Machine Learning for social sciences

Machine Learning: Session 2

William Aboucaya

Introduction

In this course, we will focus on regressions. This class of methods allows:

Introduction

In this course, we will focus on regressions. This class of methods allows:

- To **model relationships** between variables.

Introduction

In this course, we will focus on regressions. This class of methods allows:

- To **model relationships** between variables.
- To **predict outcomes** based on input data.

Introduction

In this course, we will focus on regressions. This class of methods allows:

- To **model relationships** between variables.
- To **predict outcomes** based on input data.
- To **understand how changes in input affect output**.

Introduction

In this course, we will focus on regressions. This class of methods allows:

- To **model relationships** between variables.
- To **predict outcomes** based on input data.
- To **understand how changes in input affect output**.

Regression is one of the most widely used techniques in data science, economics and social sciences.

Motivating Example

Scenario:

- You're analyzing how students perform based on different factors.
- Data collected:
  - Time spent studying in hours
  - Time spent sleeping in hours
  - Attendance at the classes
  - Exam score

Motivating Example

Scenario:

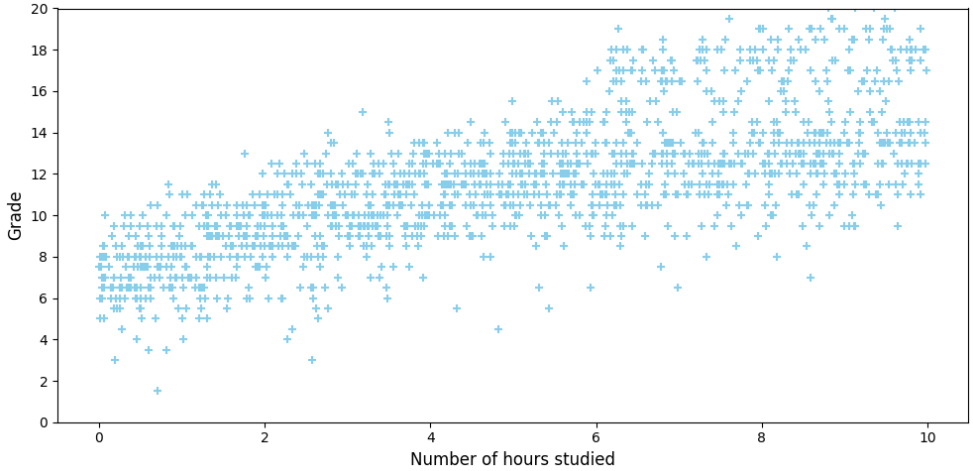
- You're analyzing how students perform based on different factors.
- Data collected:
  - Time spent studying in hours
  - Time spent sleeping in hours
  - Attendance at the classes
  - Exam score

Key Question:

If a student studies for 5 hours, attends all classes but has had a bad night's sleep before the exam, what score can we expect?

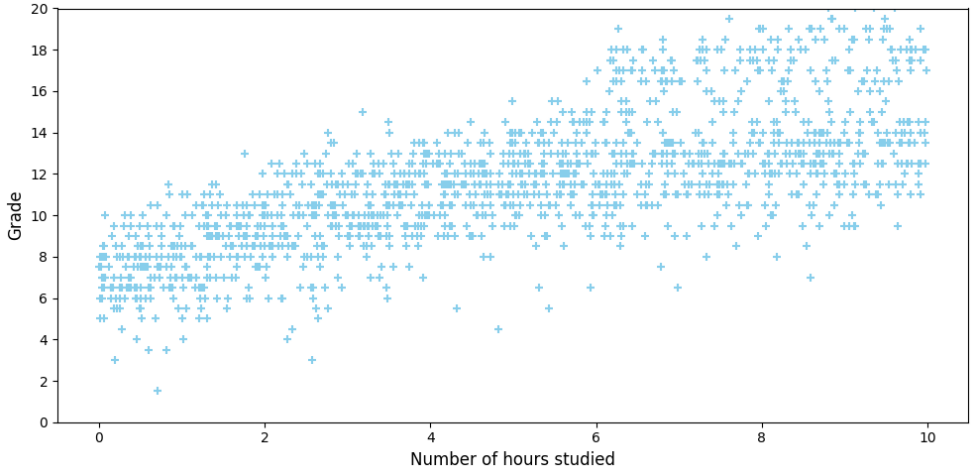
Visualizing the data

Partial representation of the data we will work with:



Visualizing the data

Partial representation of the data we will work with:



Is the relationship linear? Nonlinear? Binary?

What's Coming Up

We'll examine 3 types of regression using our example:

1. **Linear Regression** – Predicting exact score.
2. **Polynomial Regression** – Modeling nonlinear trends.
3. **Random Forest** – Capturing unspecified interactions.

Same data, different modeling goals.

Linear regression

A linear regression is a basic ML method to **predict a value** based on **one or multiple factors.**

Linear regression

A linear regression is a basic ML method to **predict a value** based on **one or multiple factors.**

The method tries to model the relationship between your features $[x_1; x_2; ...; x_n]$ and your desired output value y as a linear equation:

$$\hat{y} = \beta_0 + \beta_1 x_1 + \beta_2 x_2 + ... + \beta_n x_n$$

With βs the weights given to each factor by **fitting** your equation to the training data, and $\hat{y}$ your predicted output.

Side-note: Model training / fitting

Goal: Find the best-fitting line that minimizes the difference between the predicted and actual values.

Model:

$$\hat{y} = \beta_0 + \beta_1 x_1$$

Side-note: Model training / fitting

Goal: Find the best-fitting line that minimizes the difference between the predicted and actual values.

Model:

$$\hat{y} = \beta_0 + \beta_1 x_1$$

Loss Function (Least Squares method):

$$\text{Minimize} \quad \sum_{i=1}^{n} (y_i - \hat{y}_i)^2$$

Side-note: Model training / fitting

Goal: Find the best-fitting line that minimizes the difference between the predicted and actual values.

Model:

$$\hat{y} = \beta_0 + \beta_1 x_1$$

Loss Function (Least Squares method):

$$\text{Minimize} \quad \sum_{i=1}^{n} (y_i - \beta_0 - \beta_1 x_i)^2$$

Side-note: Model training / fitting

Goal: Find the best-fitting line that minimizes the difference between the predicted and actual values.

Model:

$$\hat{y} = \beta_0 + \beta_1 x_1$$

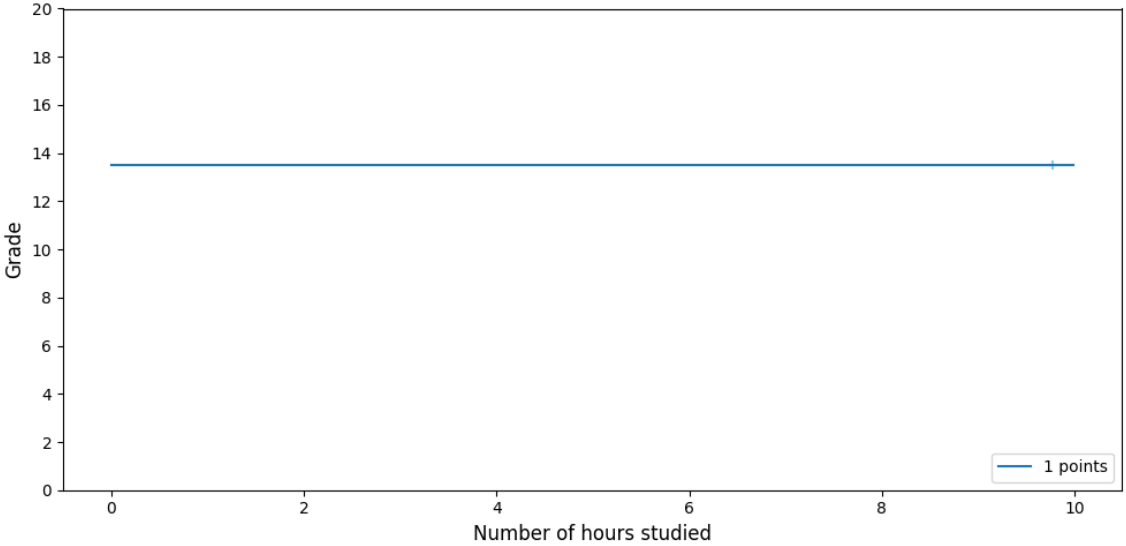
Loss Function (Least Squares method):

$$\text{Minimize} \quad \sum_{i=1}^{n} (y_i - \beta_0 - \beta_1 x_i)^2$$

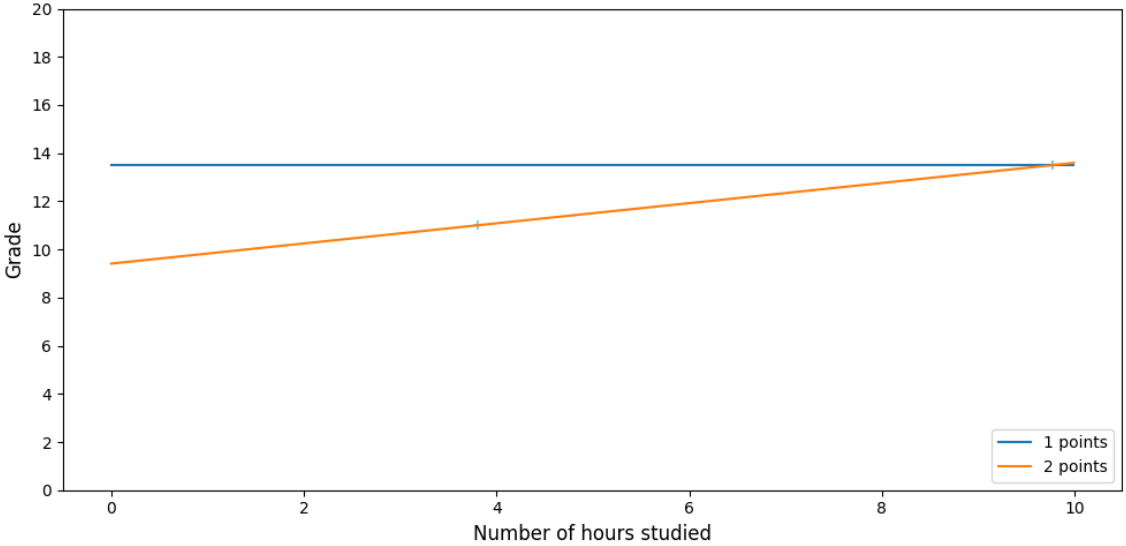
Solution: Choose β_0 and β_1 that minimize the squared error.

This class does not go into the mathematical details of how to compute your βs, we will have a Python library do it for us. But if you are interested, searching for "gradient descent" is a good first step.

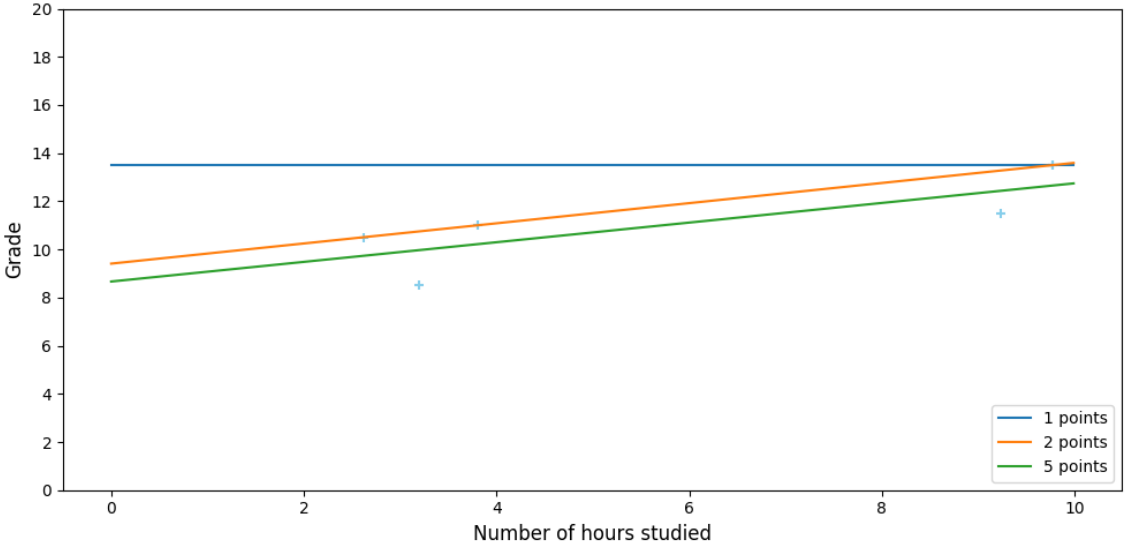
Linear regression in practice



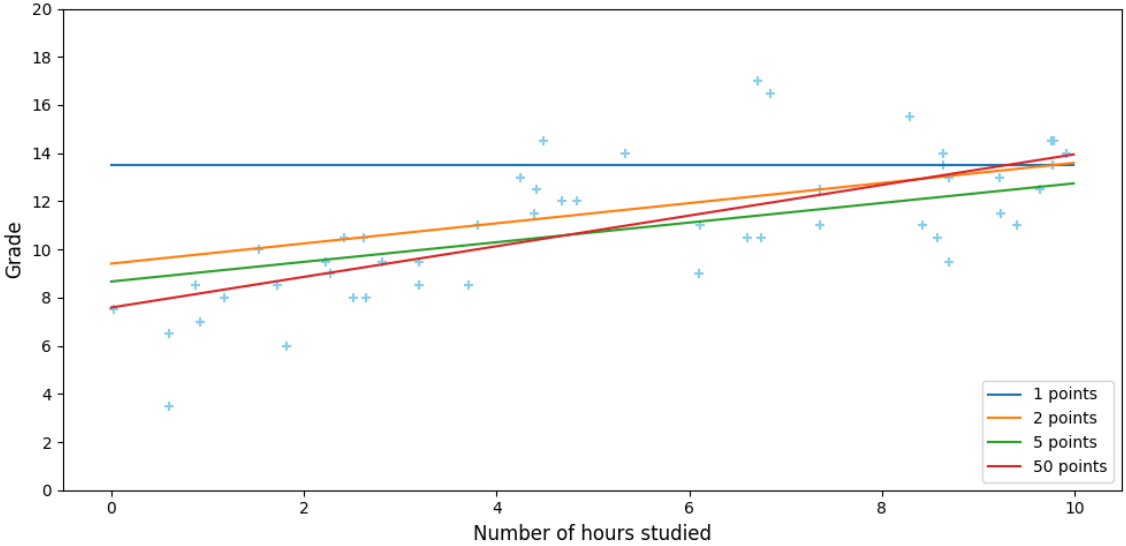
Linear regression in practice



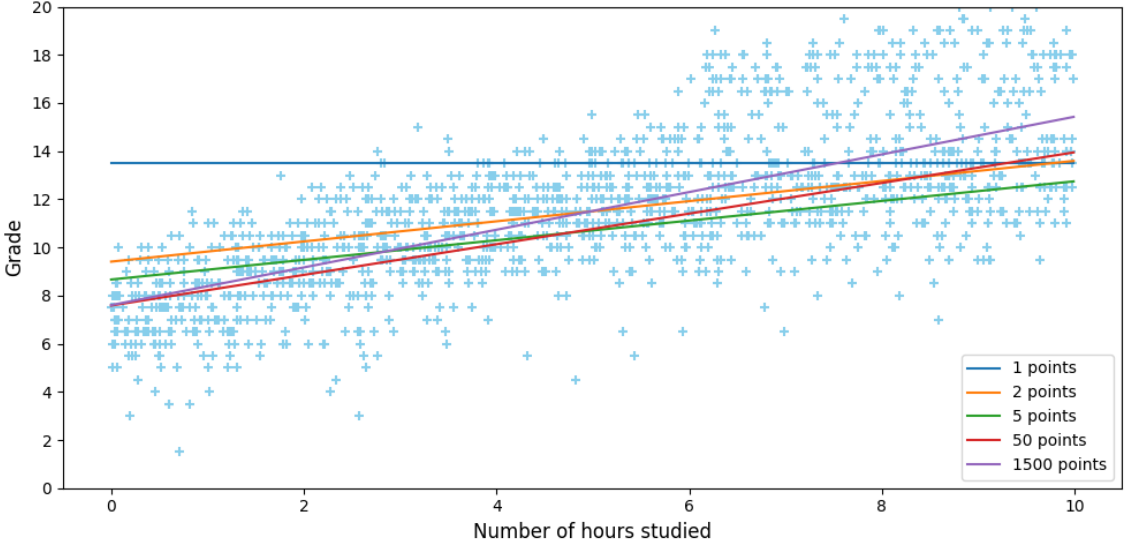
Linear regression in practice



Linear regression in practice



Linear regression in practice



Limits of linear regression

Problem: Using linear regressions is perfectly fine... to model linear relationships between independent variables and an output. But how can we model:

- Non-linear relationships?
- Relationships between the interaction of two features and an output?

Limits of linear regression

Problem: Using linear regressions is perfectly fine... to model linear relationships between independent variables and an output. But how can we model:

- Non-linear relationships?
- Relationships between the interaction of two features and an output?

Polynomial regressions and addition of **interaction terms** allow us to model these more complex relationships.

Polynomial regression

Idea: Extend linear regression by adding powers of the predictor variable(s) to model nonlinear relationships.

General Form:

$$\hat{y} = \beta_0 + \beta_1 x + \beta_2 x^2 + \beta_3 x^3 + \cdots + \beta_n x^n$$

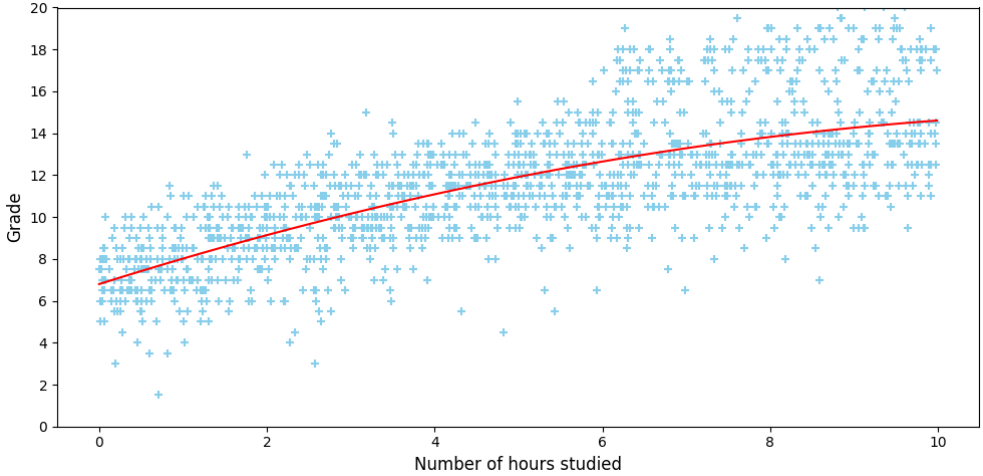
Key Points:

- Still a *linear model* in terms of the parameters β.
- Captures curves and nonlinear patterns.
- Risk of *overfitting* if degree is too high.

Polynomial regression

Example: Predict exam score from hours studied:

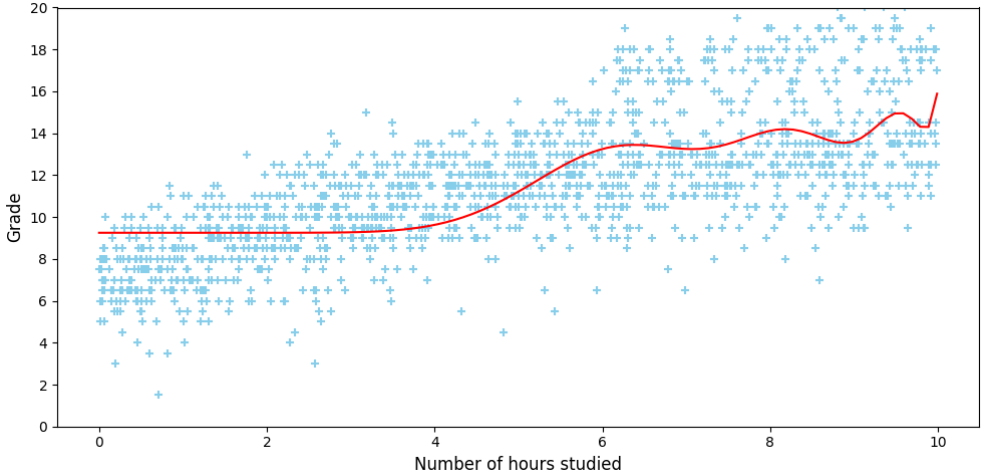
$$\text{Score} = \beta_0 + \beta_1(\text{Hours}) + \beta_2(\text{Hours})^2$$



Polynomial regression

Overfitting example by increasing the degree of the regression:

$$\text{Score} = \beta_0 + \beta_1(\text{Hours}) + \beta_2(\text{Hours})^2 + \cdots + \beta_{20}(\text{Hours})^{20}$$



Interaction terms

What are interaction terms?
Interaction terms capture the effect of two (or more) variables acting together by adding a product term. For example:

$$\hat{y} = \beta_0 + \beta_1 x_1 + \beta_2 x_2 + \beta_3 (x_1 \times x_2)$$

Interpretation: The effect of x_1 depends on the level of x_2, and vice versa.

Interaction terms

Why are they interesting?

- Real-world relationships are rarely additive.
- Capture *conditional effects* between features.
- Reveal synergistic or moderating effects.

Interactions allow more flexible modeling:

- Example: Hours studied (x_1) and sleep quality (x_2) both affect exam score.
- Their combined effect may be larger (or smaller) than the sum of their individual effects.

Interaction terms

Shortcomings:

- **Interpretability**: Interaction coefficients can be difficult to explain.
- **Complexity**: The number of possible interactions grows quickly with the number of predictors.
- **Overfitting**: Including unnecessary interactions can harm generalization.
- **Multicollinearity**: Interaction terms often correlate with main effects.

From Regression to Trees

So far: we specified polynomial terms and interaction terms *by hand* to capture nonlinearity and interactions.

From Regression to Trees

So far: we specified polynomial terms and interaction terms *by hand* to capture nonlinearity and interactions.

What if the model could discover these patterns on its own, directly from the data?

From Regression to Trees

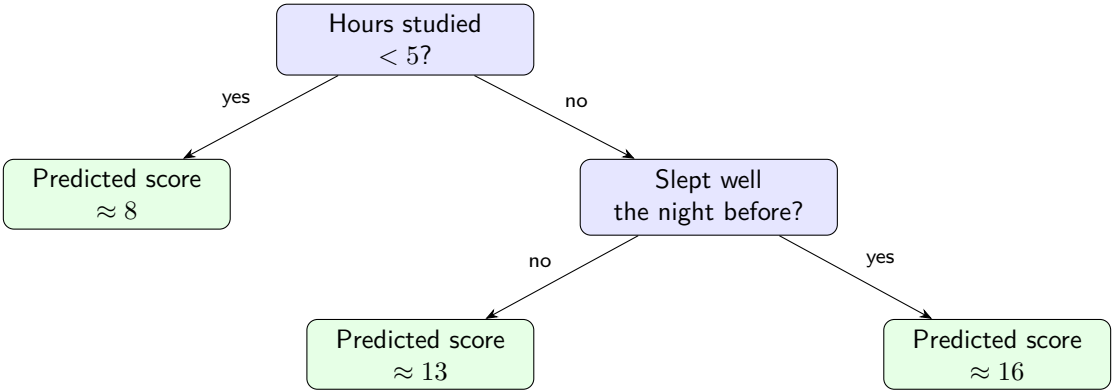
So far: we specified polynomial terms and interaction terms *by hand* to capture nonlinearity and interactions.

What if the model could discover these patterns on its own, directly from the data?

This is exactly what **tree-based methods** do – starting with the **decision tree**.

Decision trees

Idea: Repeatedly split the data based on feature values, until each group (a *leaf*) is used to make a prediction.



Decision trees: how they are built

1. Pick the feature (and threshold) that best separates the data – i.e., that most reduces prediction error.

Decision trees: how they are built

1. Pick the feature (and threshold) that best separates the data – i.e., that most reduces prediction error.
2. Split the data into two groups based on that threshold.

Decision trees: how they are built

1. Pick the feature (and threshold) that best separates the data – i.e., that most reduces prediction error.
2. Split the data into two groups based on that threshold.
3. Repeat the process recursively on each resulting group.

Decision trees: how they are built

1. Pick the feature (and threshold) that best separates the data – i.e., that most reduces prediction error.
2. Split the data into two groups based on that threshold.
3. Repeat the process recursively on each resulting group.
4. Stop once a group is small enough, or pure enough: it becomes a **leaf**, predicting the average outcome of that group.

Decision trees: strengths and limits

Strengths:

- Captures **nonlinearities** and **interactions** automatically – no manual x^2 or $x_1 \times x_2$ terms needed.
- Easy to interpret: just follow the branches.

Decision trees: strengths and limits

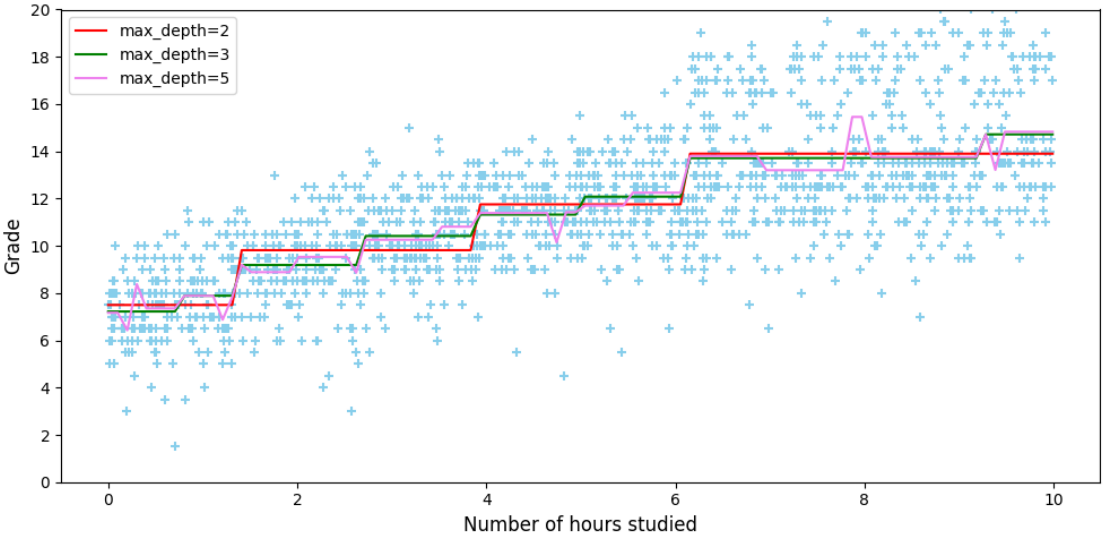
Strengths:

- Captures **nonlinearities** and **interactions** automatically – no manual x^2 or $x_1 \times x_2$ terms needed.
- Easy to interpret: just follow the branches.

Limits:

- A single tree is **unstable**: a small change in the training data can produce a very different tree.
- Prone to **overfitting**, especially when allowed to grow deep.

Predictions using a decision tree



From one tree to a forest

Idea: Instead of trusting a single, unstable tree, train **many** trees and combine their predictions.

From one tree to a forest

Idea: Instead of trusting a single, unstable tree, train **many** trees and combine their predictions.

This is called **ensemble learning**: combining multiple models usually performs better, and more reliably, than any single one on its own.

Bagging: Bootstrap Aggregating

Step 1 – Bootstrap: Create many random samples of the training data, drawn *with replacement*.

Bagging: Bootstrap Aggregating

Step 1 – Bootstrap: Create many random samples of the training data, drawn *with replacement*.

Step 2 – Train: Fit one decision tree on each sample.

Bagging: Bootstrap Aggregating

Step 1 – Bootstrap: Create many random samples of the training data, drawn *with replacement*.

Step 2 – Train: Fit one decision tree on each sample.

Step 3 – Aggregate:

- Regression: **average** the predictions of all trees.
- Classification: **majority vote** across all trees.

Bagging: Bootstrap Aggregating

Step 1 – Bootstrap: Create many random samples of the training data, drawn *with replacement*.

Step 2 – Train: Fit one decision tree on each sample.

Step 3 – Aggregate:

- Regression: **average** the predictions of all trees.
- Classification: **majority vote** across all trees.

This is the core idea behind **Random Forest**.

Random forest: adding more randomness

Bagging alone isn't quite enough: trees trained on similar bootstrap samples can still end up fairly similar to one another (**correlated**).

Random forest: adding more randomness

Bagging alone isn't quite enough: trees trained on similar bootstrap samples can still end up fairly similar to one another (**correlated**).

Random Forest adds one more trick: at each split, only a **random subset of features** is considered as a candidate.

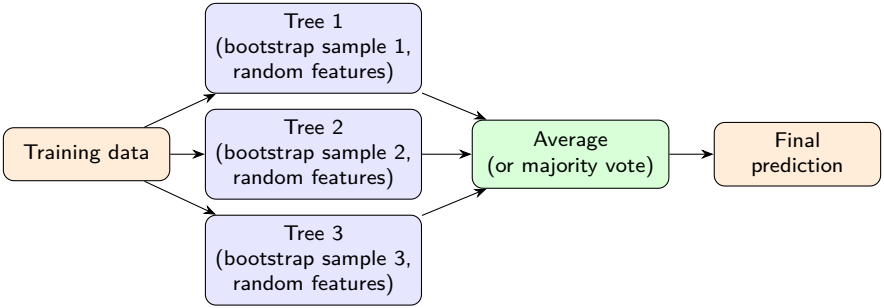
Random forest: adding more randomness

Bagging alone isn't quite enough: trees trained on similar bootstrap samples can still end up fairly similar to one another (**correlated**).

Random Forest adds one more trick: at each split, only a **random subset of features** is considered as a candidate.

Effect: trees become more different from each other, so averaging them reduces variance more effectively.

Random forest: putting it all together



(In practice, a random forest typically trains hundreds of trees, not just three.)

Feature importance

Random forests offer a natural way to answer: **which features matter most?**

Feature importance

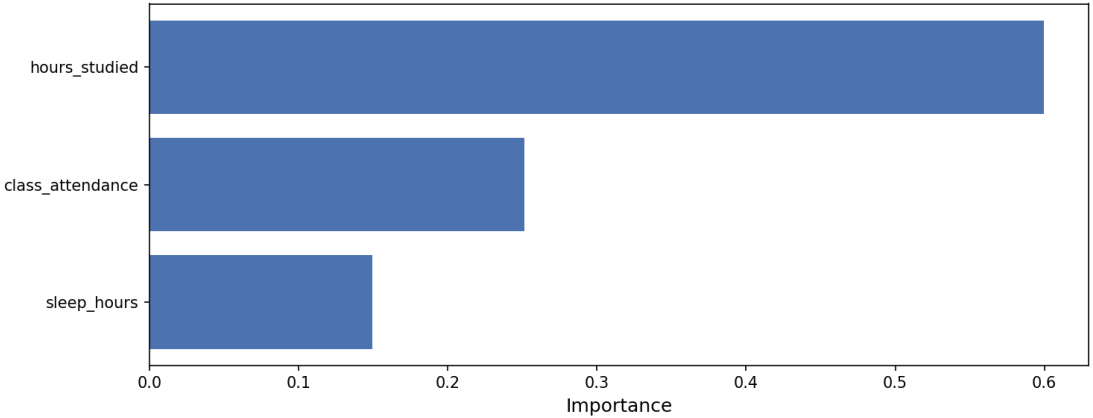
Random forests offer a natural way to answer: **which features matter most?**

For each feature, an importance score is estimated from how much it reduces prediction error, averaged across all trees and all the splits where it is used.

Feature importance

Random forests offer a natural way to answer: **which features matter most?**

For each feature, an importance score is estimated from how much it reduces prediction error, averaged across all trees and all the splits where it is used.



Random forest: strengths and limitations

Strengths:

- Captures nonlinear relationships and interactions automatically.
- Much more robust than a single tree – far less prone to overfitting.
- Provides feature importance scores for interpretability.
- Requires little manual feature engineering (no need to add x^2 or interaction terms by hand).

Random forest: strengths and limitations

Strengths:

- Captures nonlinear relationships and interactions automatically.
- Much more robust than a single tree – far less prone to overfitting.
- Provides feature importance scores for interpretability.
- Requires little manual feature engineering (no need to add x^2 or interaction terms by hand).

Limitations:

- Less directly interpretable than a regression equation – no simple β coefficients.
- Slower to train and to predict with than linear/polynomial regression.
- Cannot extrapolate beyond the range of the training data.

Back to our motivating example

If a student studies for 5 hours, attends all classes but has had a bad night's sleep before the exam, what score can we expect?

Back to our motivating example

If a student studies for 5 hours, attends all classes but has had a bad night's sleep before the exam, what score can we expect?

A random forest can capture this kind of **conditional** effect – sleep quality mattering more or less depending on hours studied – **without us having to specify it as an interaction term ourselves**.

Which regression for a given task?

Example task	Model Type	Why?
Predict raw exam score	Linear / Polynomial	Continuous output
Model optimal study hours	Polynomial	Nonlinear relationship
Predict score with complex, unspecified interactions	Random Forest	Captures nonlinearities & interactions automatically